

Project Title: Life Insurance Management System

1. Overview

The Life Insurance Management System is designed to manage life insurance policies, premium payments, claims, and customer support services. It provides functionalities for policy creation, beneficiary management, claim submission and approval, payment processing, and support ticket handling, all implemented using an MVC architecture.

This design supports both **Java (Spring MVC)** and **.NET (ASP.NET Core MVC)** frameworks.

The system consists of five primary modules:

1. Policy Management
2. Beneficiary Management
3. Claim Management
4. Payment Management
5. User Authentication & Authorization

2. Assumptions

- The system will be deployed locally using **MySQL or SQL Server**.
- Role-based access will be implemented for **admin, agent, and customer** roles.
- ORM tools (**Hibernate for Java, Entity Framework Core for .NET**) will handle database interactions.
- The UI will be responsive and optimized for desktop and mobile.

3. Module-Level Design

3.1 Policy Management Module

Purpose: Handles creation, modification, and retrieval of life insurance policies.

- **Controller:**
 - PolicyController
 - createPolicy()
 - updatePolicy()
 - getPolicyById()
 - getAllPolicies()
 - deletePolicy()
- **Service:**
 - PolicyService
 - Business logic for policy lifecycle.

- **Model:**
 - Policy Entity
 - Attributes:
 - policyId (PK)
 - policyNumber
 - coverageAmount
 - premiumAmount
 - termYears
 - startDate
 - endDate
 - policyStatus (ACTIVE, LAPSED, MATURED)

3.2 Beneficiary Management Module

Purpose: Manages beneficiaries linked to life insurance policies.

- **Controller:**
 - BeneficiaryController
 - addBeneficiary()
 - updateBeneficiary()
 - getBeneficiaryByPolicy()
 - deleteBeneficiary()
- **Service:**
 - BeneficiaryService
 - Handles beneficiary lifecycle.
- **Model:**
 - Beneficiary Entity
 - Attributes:
 - beneficiaryId (PK)
 - policyId (FK)
 - name
 - relationship
 - percentageShare

3.3 Claim Management Module

Purpose: Handles claims related to policyholder death or maturity.

- **Controller:**
 - ClaimController
 - submitClaim()
 - getClaimDetails()

- updateClaimStatus()
 - getAllClaims()
- **Service:**
 - ClaimService
 - Handles claim lifecycle and notifications.
- **Model:**
 - Claim Entity
 - Attributes:
 - claimId (PK)
 - policyId (FK)
 - claimAmount
 - claimDate
 - claimStatus (OPEN, APPROVED, REJECTED)
 - adjusterId (FK)

3.4 Payment Management Module

Purpose: Handles premium payments and refunds.

- **Controller:**
 - PaymentController
 - makePayment()
 - getPaymentDetails()
 - getPaymentsByPolicy()
- **Service:**
 - PaymentService
 - Processes payments and updates status.
- **Model:**
 - Payment Entity
 - Attributes:
 - paymentId (PK)
 - policyId (FK)
 - paymentAmount
 - paymentDate
 - paymentStatus (SUCCESS, FAILED, PENDING)

3.5 User Authentication & Authorization Module

Purpose: Provides secure login and role-based access.

- **Controller:**
 - AuthController

- login()
 - registerUser()
 - logout()
 - getUserProfile()
- **Service:**
 - AuthService
 - Handles authentication and authorization.
- **Model:**
 - User Entity
 - Attributes:
 - userId (PK)
 - username
 - password (hashed)
 - email
 - role (ADMIN, AGENT, CUSTOMER)

4. Database Schema

Policy Table

```
CREATE TABLE Policy (
  policyId INT AUTO_INCREMENT PRIMARY KEY,
  policyNumber VARCHAR(50) NOT NULL,
  coverageAmount DECIMAL(10,2),
  premiumAmount DECIMAL(10,2),
  termYears INT,
  startDate DATE,
  endDate DATE,
  policyStatus ENUM('ACTIVE','LAPSED','MATURED')
);
```

Beneficiary Table

```
CREATE TABLE Beneficiary (
  beneficiaryId INT AUTO_INCREMENT PRIMARY KEY,
  policyId INT,
  name VARCHAR(100),
  relationship VARCHAR(50),
  percentageShare DECIMAL(5,2),
  FOREIGN KEY (policyId) REFERENCES Policy(policyId)
);
```

Claim Table

```
CREATE TABLE Claim (  
  claimId INT AUTO_INCREMENT PRIMARY KEY,  
  policyId INT,  
  claimAmount DECIMAL(10,2),  
  claimDate DATE,  
  claimStatus ENUM('OPEN','APPROVED','REJECTED'),  
  adjusterId INT,  
  FOREIGN KEY (policyId) REFERENCES Policy(policyId),  
  FOREIGN KEY (adjusterId) REFERENCES User(userId)  
);
```

Payment Table

```
CREATE TABLE Payment (  
  paymentId INT AUTO_INCREMENT PRIMARY KEY,  
  policyId INT,  
  paymentAmount DECIMAL(10,2),  
  paymentDate DATE,  
  paymentStatus ENUM('SUCCESS','FAILED','PENDING'),  
  FOREIGN KEY (policyId) REFERENCES Policy(policyId)  
);
```

User Table

```
CREATE TABLE User (  
  userId INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) UNIQUE,  
  password VARCHAR(255),  
  email VARCHAR(100),  
  role ENUM('ADMIN','AGENT','CUSTOMER')  
);
```

5. Local Deployment Details

- Install **MySQL** or **SQL Server**, **JDK 17/21/24** or **.NET SDK 8.0**.
- Deploy using **Apache Tomcat (Java)** or **Kestrel (.NET)**.
- Steps:
 1. Clone repository from GitHub/Bitbucket.
 2. Set up database schema.
 3. Configure application.properties or appsettings.json.
 4. Build and run application.

6. Conclusion

The Life Insurance Management System provides a modular MVC design for managing life policies, beneficiaries, claims, and payments. It ensures secure role-based access and efficient local deployment.